

DTG TECHNOLOGY CORPORATION



INTERNSHIP REPORT

Project

Machine Learning-Based Intrusion Detection

Group: 5

Instructor: Nguyen Van Hai
Nguyen Minh Huy

Students: Do Thanh Tan

Ho Chi Minh City, September 2026



Contents

List of Figures	1
List of Tables	1
1 Introduction	3
2 Project Overview	4
2.1 Project Scope	4
2.2 Dataset and Features Scope	4
2.3 Tools and Package	5
3 Implementation Plan	6
4 Milestone and Expected Outcomes	7
5 Project Progress	8
5.1 Week 1: Initial Setup and Data Preprocessing	8
5.1.1 Challenges Encountered	8
5.1.2 Future Plans	8



List of Figures

List of Tables

1	Network flow class labels	4
2	Six-week implementation plan and expected outcomes	7
3	Number of samples for each SubCategory	9

1 Introduction

The increasing use of networked systems and Internet-connected services has led to a growing volume and complexity of network traffic monitored by security systems. Intrusion Detection Systems (IDS) play an important role in identifying potentially malicious activities and supporting network security monitoring. However, traditional rule-based and signature-based approaches can be limited when handling large-scale network telemetry or detecting patterns that have not been explicitly defined in advance.

Machine learning (ML) provides an alternative approach by learning patterns from historical network activity and applying them to new observations. In particular, supervised learning can be used to classify network events according to their security-related characteristics. However, the effectiveness of a machine-learning-based IDS depends not only on the selected algorithms but also on the quality of the data, feature selection, evaluation methodology, and the ability of the models to generalize to future network activity. This project investigates the application of basic machine learning algorithms to multiclass intrusion detection using real-world Edge IDS telemetry. Rather than treating intrusion detection as a binary classification problem between normal and malicious traffic, the project considers four main verdict classes: *benign*, *bruteforce*, *dos*, and *recon*. This multiclass formulation enables the models to distinguish between different types of network events and security-related activities.

The primary data source for this project is the GeNIS (GECAD Network Intrusion Scenarios) dataset. The dataset provides labelled network-flow data together with telemetry and enrichment information. It contains different types of network activities and attack scenarios, providing a basis for investigating machine learning methods under conditions that are closer to practical network-security monitoring. The project focuses specifically on the scenario data provided by GeNIS and develops a pipeline from data preprocessing through model training and evaluation.

Two machine learning approaches are selected for comparison: Logistic Regression and Random Forest. Logistic Regression is used as a relatively simple and interpretable linear baseline, with coefficients that can be directly examined. Random Forest represents a nonlinear ensemble approach capable of modelling more complex relationships and feature interactions. Comparing these two approaches provides a basis for examining the relationship between model complexity, interpretability, and classification performance. A key component of the project is the evaluation methodology. Because the dataset contains temporal information, the primary experiment uses a temporal train/validation/test split, in which historical observations are used for training and later observations are reserved for evaluation. This setup is intended to provide a more realistic approximation of future deployment conditions. A random stratified split is also considered as an additional experiment to examine whether random splitting may produce more optimistic performance estimates than temporal evaluation.

The project also considers potential data leakage and label provenance during the development of the pipeline. Features such as timestamps, IP addresses, and MAC addresses are excluded because they are not expected to provide useful predictive information for the intended classification task and may introduce undesirable dependencies in the models. In addition, the project investigates feature selection and the impact of different feature sets on model performance.

Overall, the project aims to develop a reproducible experimental pipeline for multiclass intrusion detection using real-world Edge IDS telemetry. The study focuses on comparing a linear baseline with a nonlinear ensemble model while examining different data-splitting strategies, feature selection, and practical issues such as data leakage and label provenance. The resulting pipeline is intended to provide a structured basis for evaluating the applicability and limitations of basic machine learning approaches to network intrusion detection.

2 Project Overview

2.1 Project Scope

The main objective of this project is to develop a reproducible machine learning pipeline for multiclass intrusion detection using real-world Edge IDS telemetry. This project focuses on the application of basic machine learning algorithms to classify network events into four distinct categories: benign, bruteforce, dos, recon. The project aims to evaluate the performance of these models under different data splitting strategies, while also considering practical issues such as data leakage and label provenance.

The project will implement and compare two machine learning algorithms: Logistic Regression and Random Forest. The primary goal is not only to obtain high classification accuracy but also to evaluate how the models behave under a realistic temporal evaluation setting and to understand the trade-offs between model interpretability

2.2 Dataset and Features Scope

This project utilizes the GeNIS (GECAD Network Intrusion Scenarios) dataset. This dataset provides multiple sequential attack scenarios, and realistic normal network activity recorded on the Airbus CyberRange platform to improve the detection and classification of cyberattacks targeting vulnerable small and medium-sized enterprises [1]. It contains features of 2,806,168 network flows that represent the traffic patterns of local and remote attackers, normal users and administrators, and background traffic of an enterprise computer network.

The recorded traffic was organized into hierarchical labels, from the most general '0' and '1' labels representing benign and malicious traffic, to the more specific categories and subcategories of each specific attack or benign activity profile. Table 1 presents an overview of the considered class labels.

Binary Label	Category Label	Subcategory Label
0	benign	benign-admin benign-background benign-user
1	bruteforce	bruteforce-ftp bruteforce-smb bruteforce-ssh
	dos	dos-hulk dos-icmp dos-pushback dos-slowloris dos-udp
	recon	recon-dns recon-nmap

Table 1: *Network flow class labels*

In this project, we will try to classify the network flows into distinct classes based on the subcategory labels, which provide a more useful understanding of the specific type of network event. The dataset contains five main folder:

- 0-info: The general information regarding the dataset.
- 1-packets: The raw packet, organized by subcategory label.
- 2-flows: The labelled flows, organized by subcategory label.
- 3-scenarios: The attack scenarios, filtered by sequential attack scenario.
- 4-preprocessed: The preprocessed flows, organized by subcategory label.

This project focuses specifically on the scenario data. Therefore, the implementation will concentrate on developing the pipeline from data preprocessing through model training and evaluation rather than processing the complete raw packet data. The main machine learning resources required by the project

are the GeNIS dataset and a computational environment capable of performing preprocessing and training on the network-flow data.

2.3 Tools and Package

The project will be implemented using Python, which provides a comprehensive ecosystem of libraries for data preprocessing, machine learning, and performance evaluation. The main tools and packages used in this project are described below.

- **Python:** The primary programming language used to implement the data preprocessing, feature selection, model training, and evaluation pipeline.
- **Pandas:** Used for loading, manipulating, cleaning, and analyzing the network-flow datasets stored in CSV format.
- **NumPy:** Used for numerical operations and efficient manipulation of feature matrices and label arrays.
- **Scikit-learn:** Used to implement the machine learning models, including Logistic Regression and Random Forest. It is also used for data splitting, feature preprocessing, class imbalance handling, and evaluation metrics.
- **Matplotlib:** Used to visualize model performance, including confusion matrices and other evaluation results.
- **Jupyter Notebook:** Used during the data investigation and exploratory analysis stages to examine the dataset, analyze feature distributions, and validate preprocessing procedures.
- **Git:** Used for version control and to maintain a reproducible implementation history throughout the project.

The preprocessing and model training procedures will be implemented using Scikit-learn pipelines where appropriate. This helps ensure that preprocessing operations, such as feature scaling and encoding, are applied consistently to the training and testing data while reducing the risk of data leakage. The exact package versions and execution environment will be documented to support reproducibility.

3 Implementation Plan

Based on the current project scope, the individual work will follow the main stages of the machine learning pipeline:

1. **Dataset investigation:** examine the structure of the GeNIS dataset, including its scenario data and available network-flow features.
2. **Data preprocessing:** prepare the scenario data for machine learning and determine the features that should be included or excluded from the experiments.
3. **Feature selection:** investigate the contribution of different features and exclude features that may introduce data leakage or are not suitable for the intended deployment setting.
4. **Model implementation:** implement Logistic Regression and Random Forest for multiclass network-event classification.
5. **Experimental evaluation:** evaluate the models using the temporal train/validation/test split and the additional random stratified split.
6. **Analysis:** compare the behavior of the two models under the different evaluation strategies and investigate the effect of feature selection and data splitting on the obtained results.
7. **Reproducibility and reporting:** organize the preprocessing, training, and evaluation procedures into a reproducible experimental pipeline and document the findings.

This plan follows the project's defined workflow from dataset preparation through model training and evaluation.

4 Milestone and Expected Outcomes

The project is planned to be completed within six weeks. The implementation follows a sequential workflow, starting from understanding and preparing the GeNIS dataset, followed by feature selection and model development, and finally conducting experiments, analyzing the results, and completing the reproducible pipeline and project report. Each week has a specific objective and corresponding expected outcomes.

Week	Implementation Activities	Main Objectives	Expected Outcomes
Week 1	Study the project requirements and investigate the GeNIS dataset. Examine the dataset organization, scenario data, network-flow features, and hierarchical class labels.	Establish a clear understanding of the dataset and define the input and target variables for the project.	Dataset structure and class-label documentation; initial project setup; clearly defined classification task.
Week 2	Prepare the scenario data for machine learning. Inspect the available features, identify unsuitable features, investigate potential data leakage, complete feature selection, and define the temporal train/validation/test split and random stratified split.	Establish a clean dataset and a complete, reproducible data preparation and evaluation strategy.	Preprocessed dataset; documented feature exclusions; final feature set; reproducible data-splitting procedure; training, validation, and testing datasets.
Week 3	Implement and train the Logistic Regression baseline. Verify the training pipeline and evaluate the model using the defined experimental setup.	Establish a baseline for multi-class intrusion detection performance.	Working Logistic Regression pipeline and initial evaluation results.
Week 4	Implement and train the Random Forest model. Apply the same experimental setup used for Logistic Regression.	Evaluate the nonlinear ensemble approach and obtain comparable model results.	Working Random Forest pipeline and initial evaluation results.
Week 5	Conduct the complete experiments using both temporal and random stratified splitting strategies. Investigate the effect of different feature selections on model performance.	Analyze model behavior under different evaluation conditions and identify potential differences between realistic and random evaluation settings.	Complete experimental results for both models and both splitting strategies; feature-impact analysis.
Week 6	Analyze and compare the experimental results. Examine the trade-offs between Logistic Regression and Random Forest, investigate potential data leakage issues, organize the complete preprocessing, training, and evaluation workflow, and finalize the project documentation and report.	Consolidate the experimental findings, ensure pipeline reproducibility, and prepare the final project deliverables.	Final experimental analysis; reproducible end-to-end pipeline; documented findings and limitations; completed report and final project deliverables.

Table 2: *Six-week implementation plan and expected outcomes*

The implementation plan is structured so that each week produces a concrete output that supports the following stage. Week 1 focuses on understanding the project requirements and dataset. Week 2 combines data preprocessing, feature selection, leakage investigation, and the definition of the experimental splitting strategies. Weeks 3 and 4 focus on implementing and evaluating Logistic Regression and Random Forest, respectively. Week 5 is dedicated to conducting the complete experiments and investigating the impact of feature selection and data-splitting strategies. Finally, Week 6 focuses on consolidating the results, ensuring reproducibility, and completing the final documentation and project deliverables.

5 Project Progress

This section presents the progress of the project, including the tasks completed, challenges encountered, and planned future work.

5.1 Week 1: Initial Setup and Data Preprocessing

After selecting the GeNIS dataset, I began by investigating its structure, feature distributions, and class labels. The dataset contains over 37 million packets and more than 100 different features, including network-flow statistics, protocol information, and other traffic-related attributes.

In this project, a 30-second flow interval was selected for network traffic analysis. This choice was motivated by the temporal characteristics of certain attack scenarios, where some attacks may last approximately 60 seconds. A 30-second interval allows such an attack period to be represented by multiple temporal segments, potentially providing more detailed information about changes in traffic behavior during the attack.

In addition, the selected interval provides a balance between temporal resolution and computational efficiency. Shorter intervals may provide more fine-grained traffic information but can increase the number of observations and computational requirements. In contrast, longer intervals may aggregate traffic over a broader period and potentially obscure short-term behavioral changes.

Therefore, the 30-second interval was selected as a practical temporal setting for investigating network traffic patterns and supporting the subsequent machine learning experiments.

Due to the large number of available features, I selected a subset of features that were considered relevant to the network intrusion detection task. This feature selection step was performed to reduce the dimensionality of the input data, simplify the preprocessing pipeline, and improve computational efficiency during model training. Features that were not considered useful for the classification task were excluded from the initial feature set.

The selected features consisted of both numerical and categorical variables. Numerical features, such as packet counts, byte counts, and traffic rates, were retained in their numerical form because they represent quantitative characteristics of network traffic.

For categorical features, one-hot encoding was applied to **State**, **Flgs**, and **Proto**. These features represent categorical properties of network flows, and their values do not necessarily have an inherent numerical ordering. Therefore, directly converting their categories into integer values could introduce an artificial relationship between categories. One-hot encoding transforms each category into a separate binary feature, allowing the machine learning models to process categorical information without imposing an ordinal relationship.

The preprocessing procedure also included converting numerical columns into appropriate numeric formats and handling invalid values. The processed features were then separated from the target variable, **SubCategoryLabel**, which represents the specific type of network activity.

After preprocessing, the dataset was divided into training and testing subsets using a stratified 80:20 split. Stratification was applied to preserve the relative distribution of the classes in both subsets. A fixed random state was used to improve the reproducibility of the experiment.

5.1.1 Challenges Encountered

One of the main challenges identified during the data investigation was class imbalance. The dataset contains a substantial difference in the number of samples across different network-flow subcategories. Some classes contain a large number of samples, whereas others are represented by significantly fewer observations. (See Table 3 for the number of samples for each **SubCategory**.)

This imbalance may cause machine learning models to favor majority classes during training, potentially reducing their ability to correctly identify minority classes. Consequently, overall accuracy alone may not provide a sufficient representation of model performance, particularly for less frequent intrusion categories.

5.1.2 Future Plans

To address the class imbalance problem, I plan to investigate two different approaches during the model-training stage.

Class	Number of Samples	Percentage
dos-udp	131072	16.05
dos-icmp	131072	16.05
dos-pushack	130942	16.04
user	125128	15.33
dos-slowloris	85212	10.44
background	83244	10.20
dos-hulk	51327	6.29
admin	32664	4.00
recon-nmap	27713	3.39
bruteforce-smb	10001	1.22
bruteforce-ssh	4696	0.58
bruteforce-ftp	3344	0.41
recon-dns	20	0.00
Total	816435	100.00

Table 3: *Number of samples for each SubCategory*

The first approach is class weighting. The `class_weight='balanced'` option will be applied to assign greater importance to underrepresented classes during model training. This approach retains the original training samples while adjusting the contribution of each class to the learning process.

The second approach is random undersampling. In this approach, the number of samples from majority classes will be reduced according to a predefined sampling strategy. This is intended to reduce the dominance of majority classes and decrease the computational cost of model training.

Both approaches will be evaluated using Logistic Regression and Random Forest, resulting in four experimental configurations. Their performance will be compared using per-class Precision, Recall, F1-score, Macro F1-score, Weighted F1-score, Balanced Accuracy, and confusion matrices.

The results will be used to investigate how different imbalance-handling strategies affect multiclass intrusion detection performance, particularly for minority classes.

References

- [1] Miguel Silva, Daniela Pinto, João Vitorino, José Gonçalves, Eva Maia, and Isabel Praça. Genis: A modular dataset for network intrusion detection and classification. *Data in Brief*, 60:111487, 2025.